



Микросервисная платформа для продажи спутниковых продуктов

ORBITAMARKET



Разработка платформы с поддержкой заказов,
внутреннего биллинга в геокредитах и асинхронной
оплаты через брокер сообщений



Куликова Алёна Владимировна

2026 год

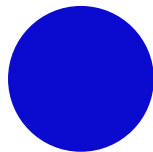
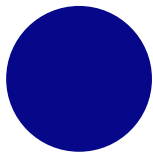
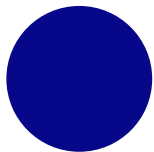
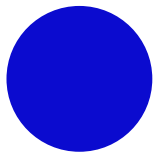


О ПРОЕКТЕ

**ЭТАПЫ И СРОКИ
РАЗРАБОТКИ**

**ВЗАИМОДЕЙСТВИЕ
МИКРОСЕРВИСОВ**

**ПРОБЛЕМЫ
БЕЗОПАСНОСТИ**



**ОСОБЕННОСТИ
РЕАЛИЗАЦИИ**

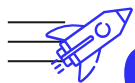
**СОБСТВЕННЫЕ
ИДЕИ**

**ПРОТЕСТИРОВАННЫЕ
СЦЕНАРИИ**

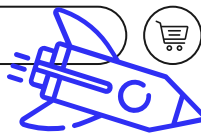
**ПЛАН
ДОРАБОТКИ
ДО MVP**

ДАЛЕЕ ИНДИКАЦИЯ





О ПРОЕКТЕ



ORBITAMARKET

это микросервисная платформа
для оформления заказов на
спутниковые продукты



Архивные снимки (ARCHIVE)
Заказ на съемку (TASKING)
Мониторинг территории (MONITORING)



БИЗНЕС- ЦЕННОСТЬ

- Пользователи могут создавать счет в геокредитах, пополнять баланс, оформлять заказы и отслеживать статус оплаты.
- Все платежи проходят асинхронно через брокер сообщений, что обеспечивает высокую надёжность и устойчивость к сбоям.

ЦЕЛЕВАЯ АУДИТОРИЯ

- Клиенты платформы (покупатели спутниковых данных)
- Операторы платформы
- Разработчики / команда сопровождения



ОСОБЕННОСТИ РЕАЛИЗАЦИИ



АРХИТЕКТУРНЫЙ ПОДХОД

	ТЕХНОЛОГИЯ	НАЗНАЧЕНИЕ
API Gateway	Spring Cloud Gateway	Единая точка входа, маршрутизация /payments/*, /orders/*
Orders Service	Spring Boot 3.2.0	Управление заказами, валидация, статусы
Payments Service	Spring Boot 3.2.0	Управление счетами, баланс, списание геокредитов
Message Broker	Apache Kafka	Асинхронное взаимодействие, гарантированная доставка
Базы данных	PostgreSQL	orders_db, payments_db



ТЕХНИЧЕСКИЕ ОСОБЕННОСТИ

- Защита от повторного списания через Inbox/Outbox таблицы
- Пессимистичная блокировка @Lock(LockModeType.PESSIMISTIC_WRITE) для конкурентного доступа к балансу
- Атомарное сохранение заказа и события
- Асинхронность — оплата через Kafka без блокировки пользователя
- Единый формат ошибок — {"error_code": "...", "message": "...", "timestamp": "..."}



ДОРОЖНАЯ КАРТА

ЭТАПЫ И СРОКИ РАЗРАБОТКИ



Подготовка проекта
и архитектуры

10 июня –
12 июня

Реализация
Orders Service

18 июня –
23 июня

Реализация API
Gateway

28 июня –
29 июня

SQL-аналитика

2 июля

Тестирование

5 июля –
7 июля

Подготовка
презентации и демо

9 июля –
10 июля

Реализация
Payments Service

13 июня –
17 июня

Настройка брокера
и асинхронности

24 июня –
27 июня

Инфраструктура и
docker-compose

30 июня –
1 июля

Диаграммы и
документация

3 июля –
4 июля

Информационная
безопасность

8 июля

Финальная
проверка и сдача

11 июля



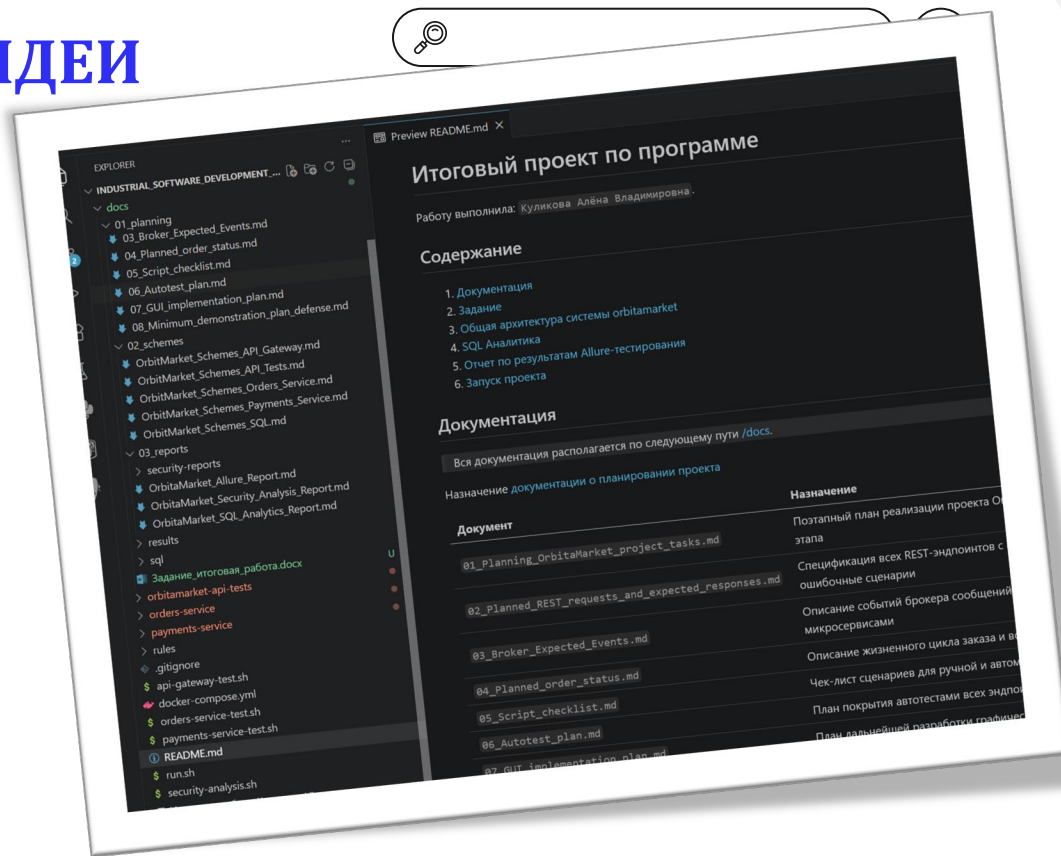
СОБСТВЕННЫЕ ИДЕИ

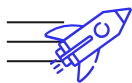


- Скрипты по запуску .sh
- Разбивка на документы в /docs
- В скриптах установка недостающих зависимостей



- Не хватило времени на реализацию GUI





ВЗАИМОДЕЙСТВИЕ МИКРОСЕРВИСОВ



ПОТОК СОЗДАНИЯ ЗАКАЗА

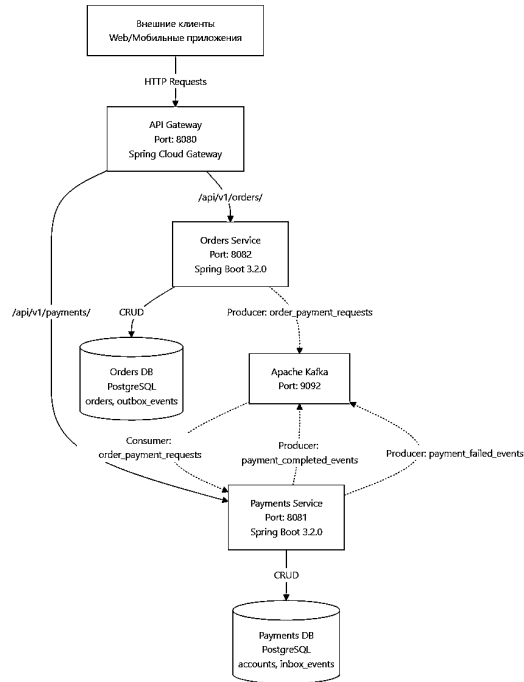
синхронная часть

- Клиент⇒API Gateway: POST /api/v1/orders с X-User-Id
- Gateway⇒Orders Service
- Orders Service: валидация ⇒ сохранение заказа ⇒ сохранение OutboxEvent ⇒ обновление статуса (PAYMENT_PENDING)
- Orders Service ⇒ клиент: 201 Created с order_id

OUTBOX WORKER

асинхронная часть

- Каждые 5 секунд OutboxWorker забирает PENDING-события
- Отправляет в Kafka топик order_payment_requests
- Payments Service получает событие ⇒ проверяет Inbox → списывает баланс ⇒ отправляет результат





ПРОТЕСТИРОВАННЫЕ СЦЕНАРИИ

АВТОТЕСТЫ

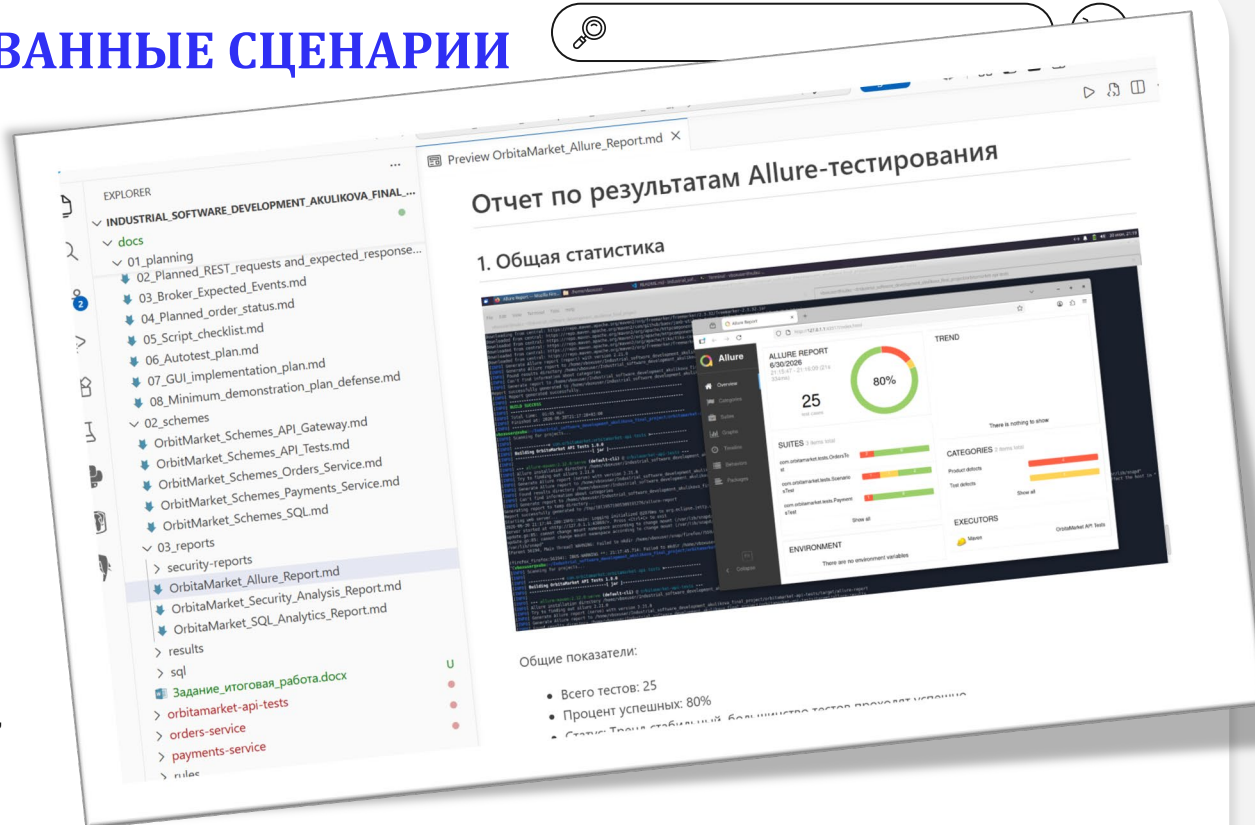
- JUnit 5
- RestAssured
- Allure

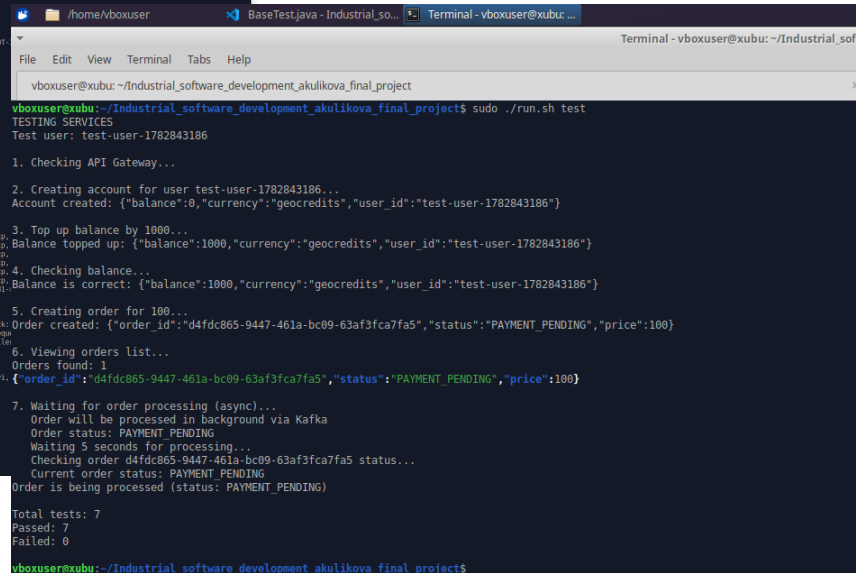
СТАТИСТИКА ALLURE

- Всего тестов: 25
- Процент успешных: 80%
- Тренд стабильный

ТЕСТ-КЕЙСЫ

- Создание счета, пополнение, баланс, валидация, ошибки (Payments Service)
- Создание заказа, валидация, список, получение по ID (Orders Service)
- Успешная оплата, недостаточно средств, два заказа при одном балансе, повторное создание счета (End-to-End сценарии)

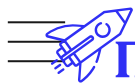




```
$ ./run.sh test
```

```
Total tests: 7
Passed: 7
Failed: 0

vboxuser@xubu:~/Industrial software development akulikova final projects$
```



ПРОБЛЕМЫ БЕЗОПАСНОСТИ



EXPLORER

INDUSTRIAL SOFTWARE DEVELOPMENT ...

api-gateway

docs

01_planning

01

01_Planning_OrbitaMarket_project_tasks.md

02_Planned_REST_requests_and_expected_response...

03_Broker_Expected_Events.md

04_Planned_order_status.md

05_Script_checklist.md

06_Autotest_plan.md

07_GUI_implementation_plan.md

08_Minimum_demonstration_plan_defense.md

02_schemes

OrbitaMarket_Schemes_API_Gateway.md

OrbitaMarket_Schemes_API_Tests.md

OrbitaMarket_Schemes_Orders_Service.md

OrbitaMarket_Schemes_Payments_Service.md

OrbitaMarket_Schemes_SQL.md

03_reports

security-reports

OrbitaMarket_Allure_Report.md

OrbitaMarket_Security_Analysis_Report.md

OrbitaMarket_SQL_Analytics_Report.md

results

allure

sql

run_sh_test.png

run_sh.png

security-analysis.png

sql

Preview OrbitaMarket_Security_Analysis_Report.md

3. Общая таблица

Таблица триажа

ID	Инструмент	Находка	Критичность	TP / FP / Риск	Решение
1	Semgrep	Отсутствие Rate Limiting в <code>OrderController</code>	Низкая	TP (Risk Accepted)	Добавить <code>@RateLimiter</code> или использовать <code>Resilience4j</code> для защиты эндпоинтов. В текущей MVP-версии риск DoS приемлем. Запланировать на доработку.
2	Semgrep	Race Condition (check-then-act) при проверке <code>userId</code> в <code>OrderController</code>	Средняя	FP	Метод контроллера выполняется в одном потоке на запрос. Проверка заголовка происходит до вызова сервиса, состояние гонки невозможно в рамках одного HTTP-запроса.
3	Semgrep	Отсутствие идемпотентности при создании заказа	Высокая	TP	Реализовать механизм идемпотентности через <code>Idempotency-Key</code> в заголовке запроса. Хранить ключи в Redis или БД с TTL. Приоритет - высокий.
4	Semgrep	Перехват общего исключения <code>Exception</code> в <code>OrderController</code>	Низкая	TP (Risk Accepted)	Заменить на конкретные исключения (<code>IllegalArgumentException</code> , <code>JsonProcessingException</code> , <code>DataAccessException</code>) для корректной обработки. В текущей версии приемлемо.
5	Semgrep	Неполный switch по <code>ProductType</code> в <code>OrderService</code>	Низкая	TP	Добавить <code>default</code> ветку или убедиться, что все enum значения обработаны. В текущей реализации только три типа, но для безопасности добавить <code>default</code> с <code>throw</code> .
6	Semgrep	Отсутствие Rate Limiting в <code>AccountController</code>	Низкая	TP (Risk Accepted)	Аналогично Orders Service. Эндпоинты пополнения баланса особенно чувствительны. Запланировать внедрение ограничений.
7	Semgrep	Race Condition (check-then-act) при проверке <code>amount</code> в <code>AccountController</code>	Средняя	FP	Проверка выполняется в контроллере, метод сервиса использует пессимистичную блокировку (<code>@Lock(LockModeType.PESSIMISTIC_WRITE)</code>). Состояние гонки исключено на уровне БД.
8	Semgrep	Перехват общего исключения <code>Exception</code> в <code>PaymentProcessor</code>	Средняя	TP	В Kafka listener критично обрабатывать конкретные исключения. Перехват <code>Exception</code> может скрыть серьезные проблемы. Рекомендуется разделить обработку на <code>JsonProcessingException</code> , <code>DataAccessException</code> и т.д.



OrbitaMarket_Security_Analysis_Report.md



ПЛАН ДОРАБОТКИ ДО MVP



01

Внедрить Idempotency -
Key для создания заказа

02

Добавить Rate Limiting

03

Заменить перехват Exception
на конкретные исключения

04

Добавить поддержку
TASKING и MONITORING

05

Реализовать GUI

06

Настроить мониторинг
(Prometheus + Grafana)

07

Добавить
аутентификацию





ЗАКЛЮЧЕНИЕ



Разработана микросервисная платформа OrbitaMarket с полным циклом заказа и оплаты.



Реализован надежный асинхронный механизм оплаты через Kafka с защитой от дублирования.



Проведено тестирование (Allure) и анализ безопасности (Gitleaks + Semgrep).



Проект готов к демонстрации и дальнейшему развитию.



https://github.com/Kulikova-A18/Industrial_software_development_akulikova_final_project



СПАСИБО ЗА ВНИМАНИЕ!

В данной презентации было 20 